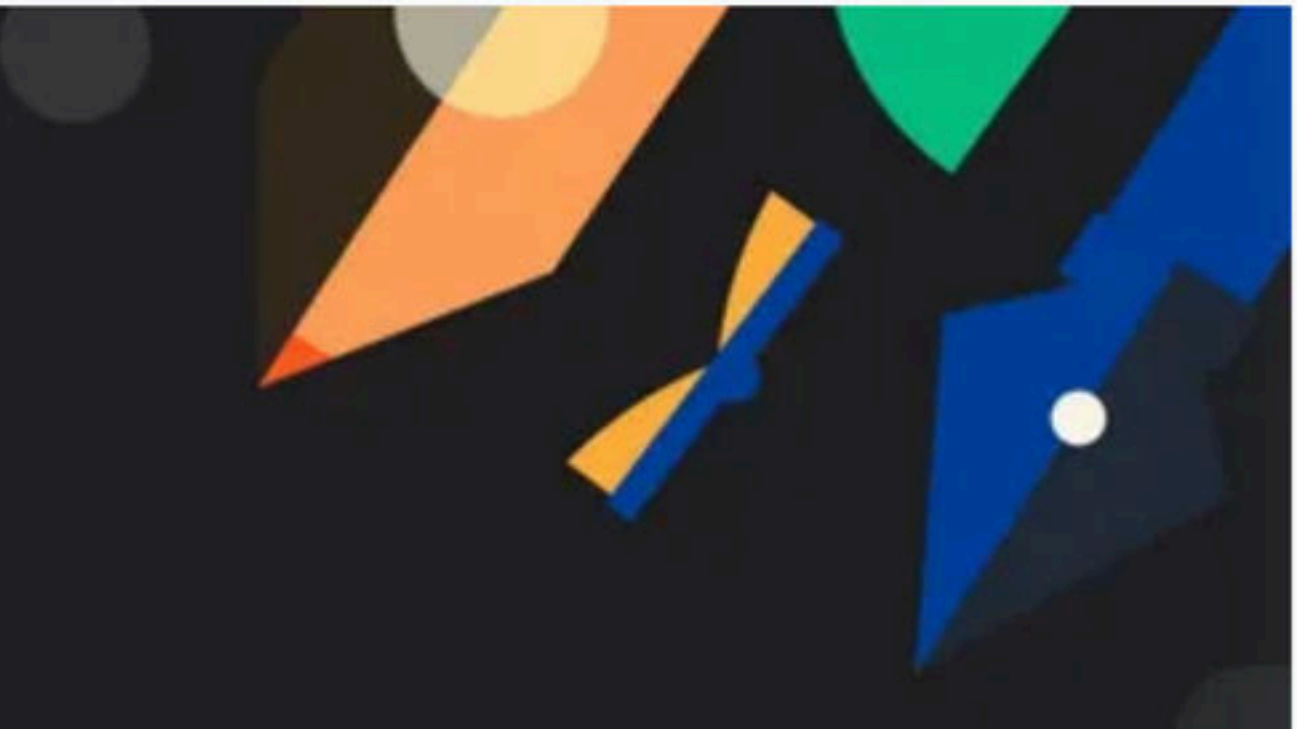




L1 - Linked Lists

Special class

Linked List →

1 2 is Hindi

Linked

List

1 lkn

① S

index

access

~~$O(1)$~~

index

access

$O(1)$

Non-continuous
memory

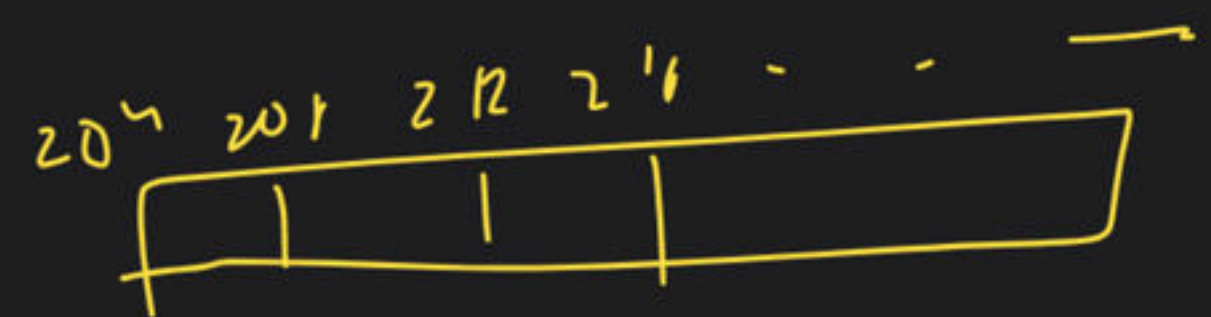
array

arr[111]

20 → int

20 × 4

80 byte



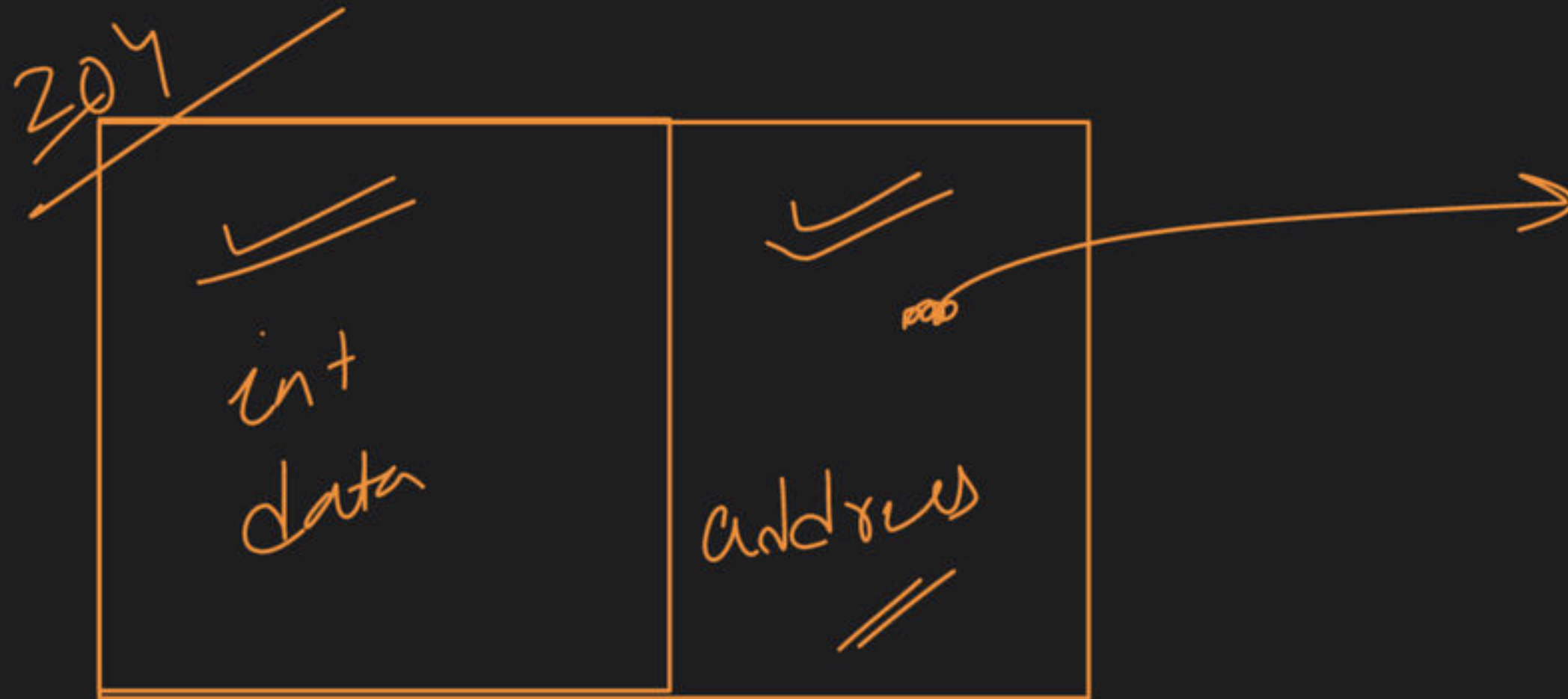
Non-continuous

1.0/0

99.1.
2

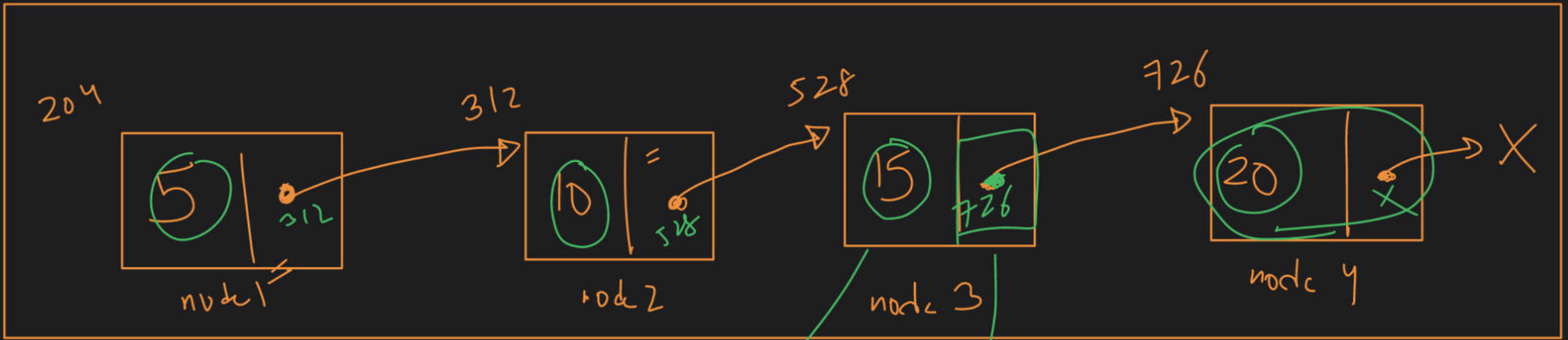
Linked List $\rightarrow$ collection of Nodes $\rightarrow \times$

Simplest
form of
Node

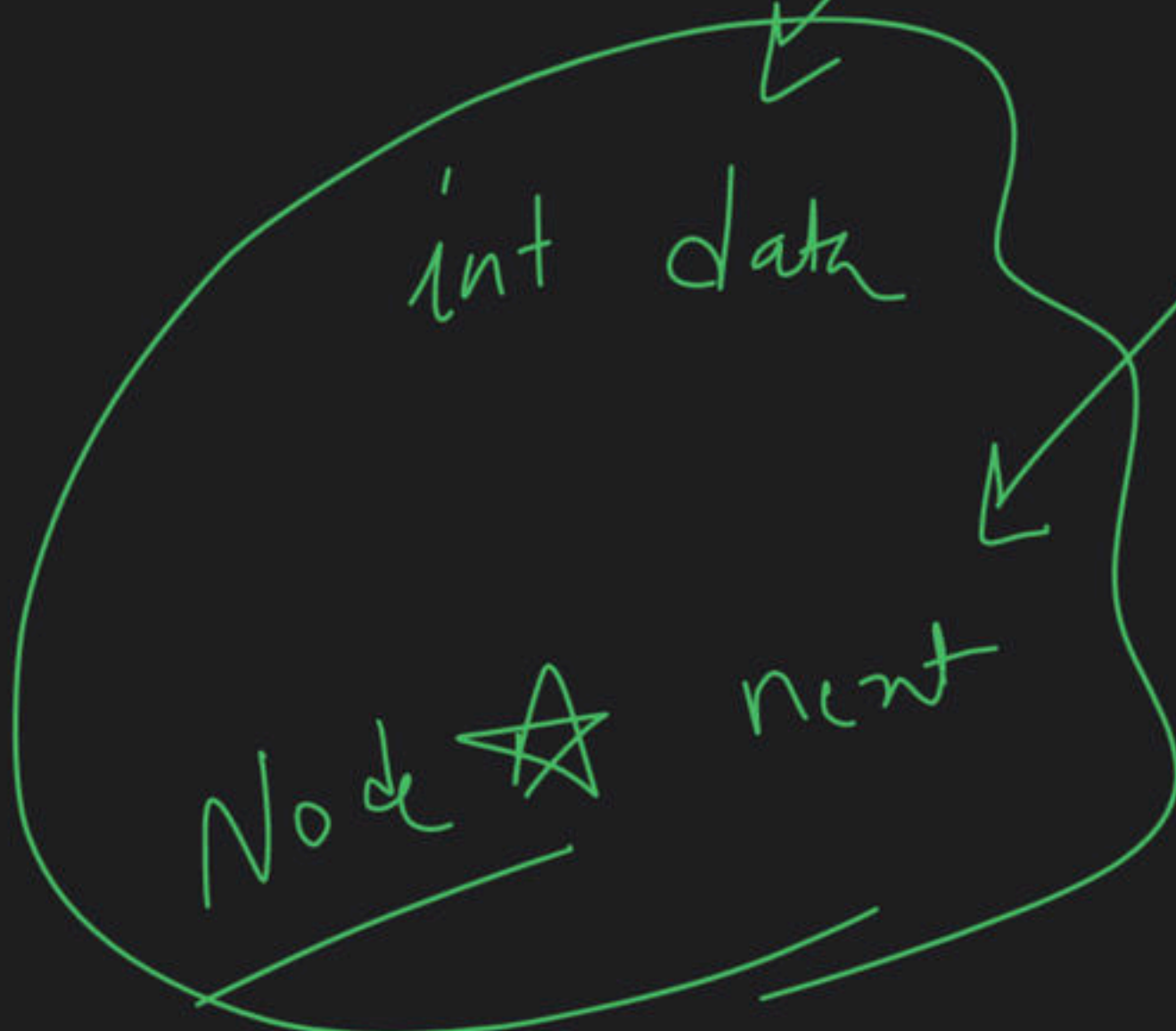


Node



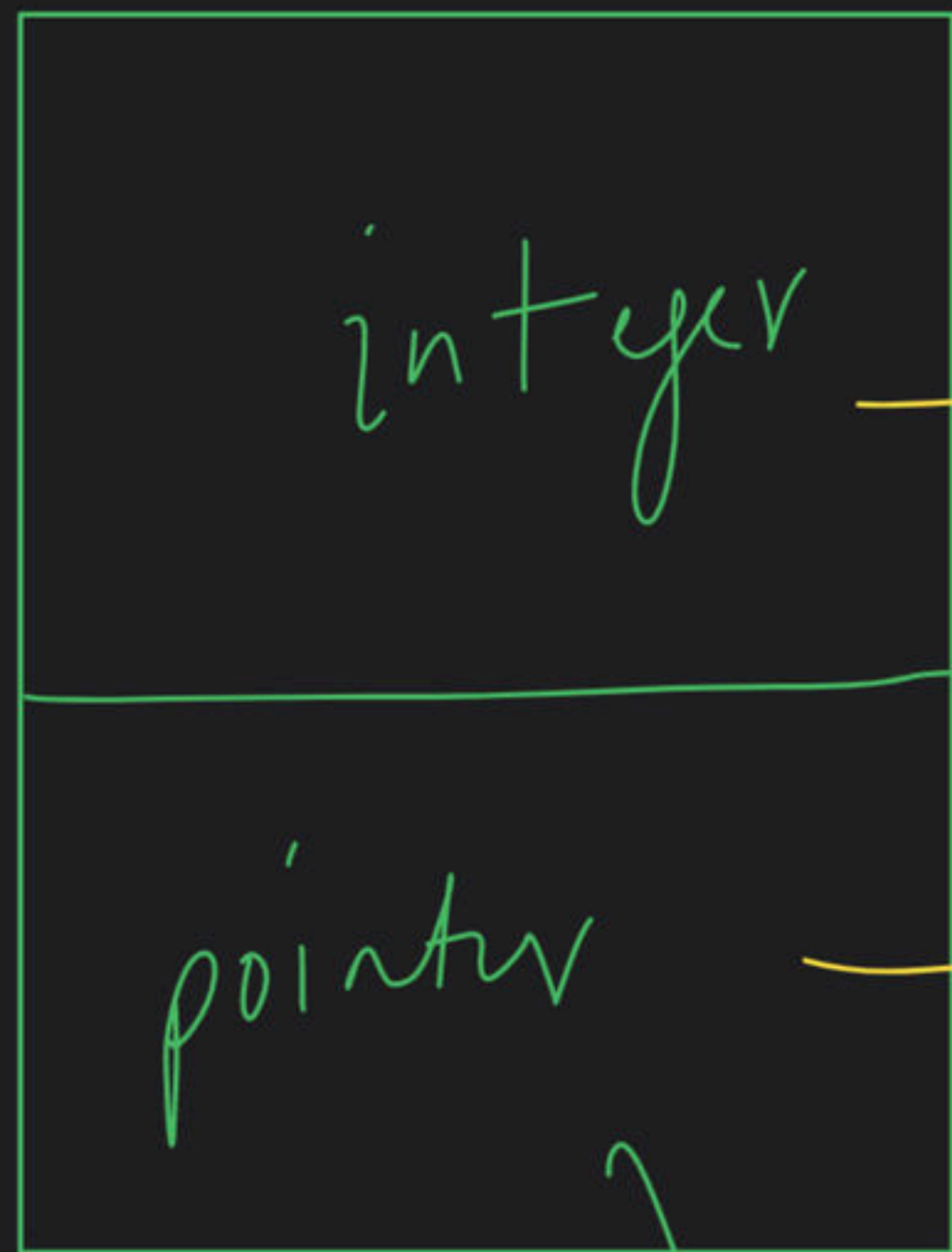


Linked list



int *
bool *
float *

Node



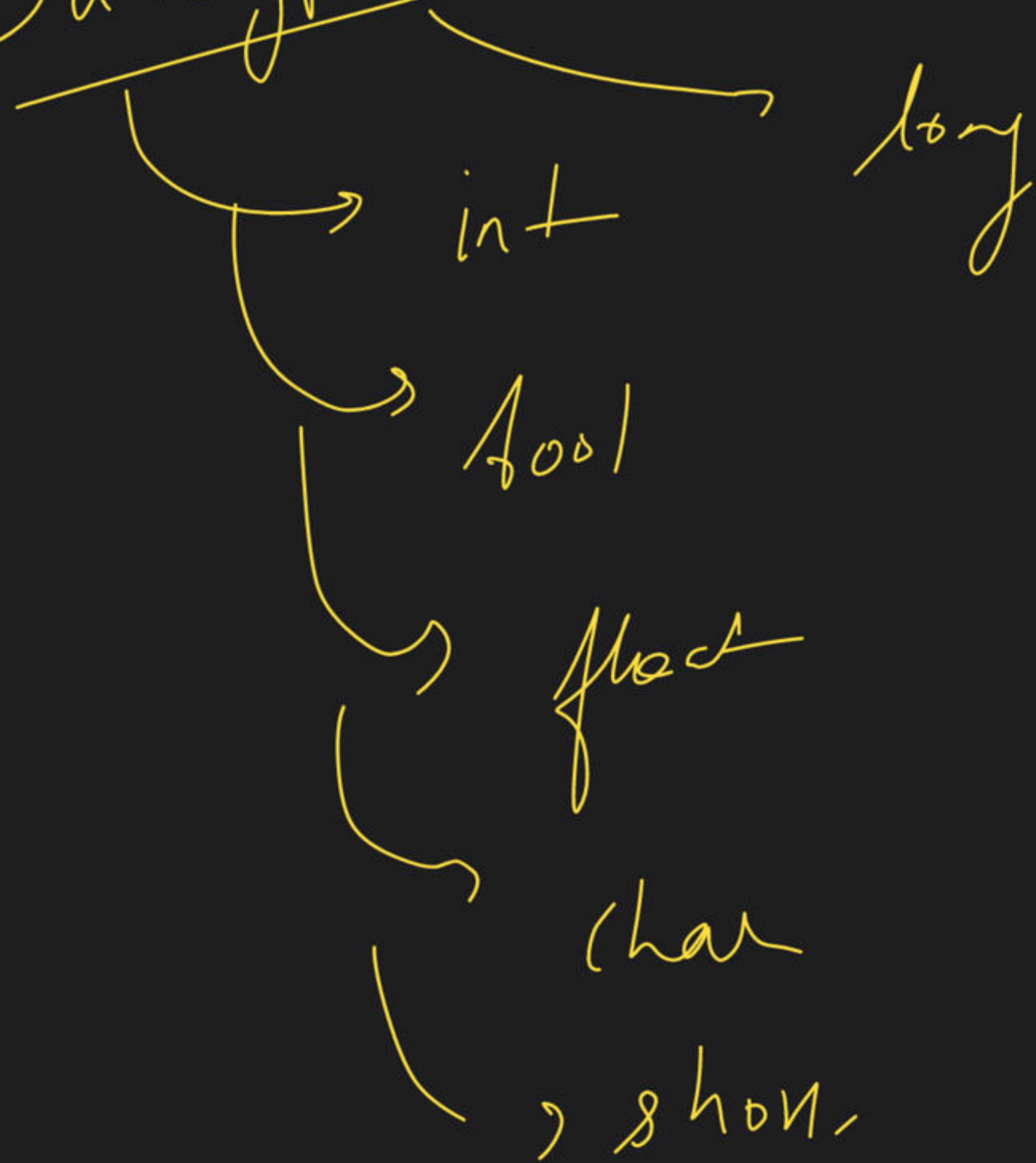
integer

int data;

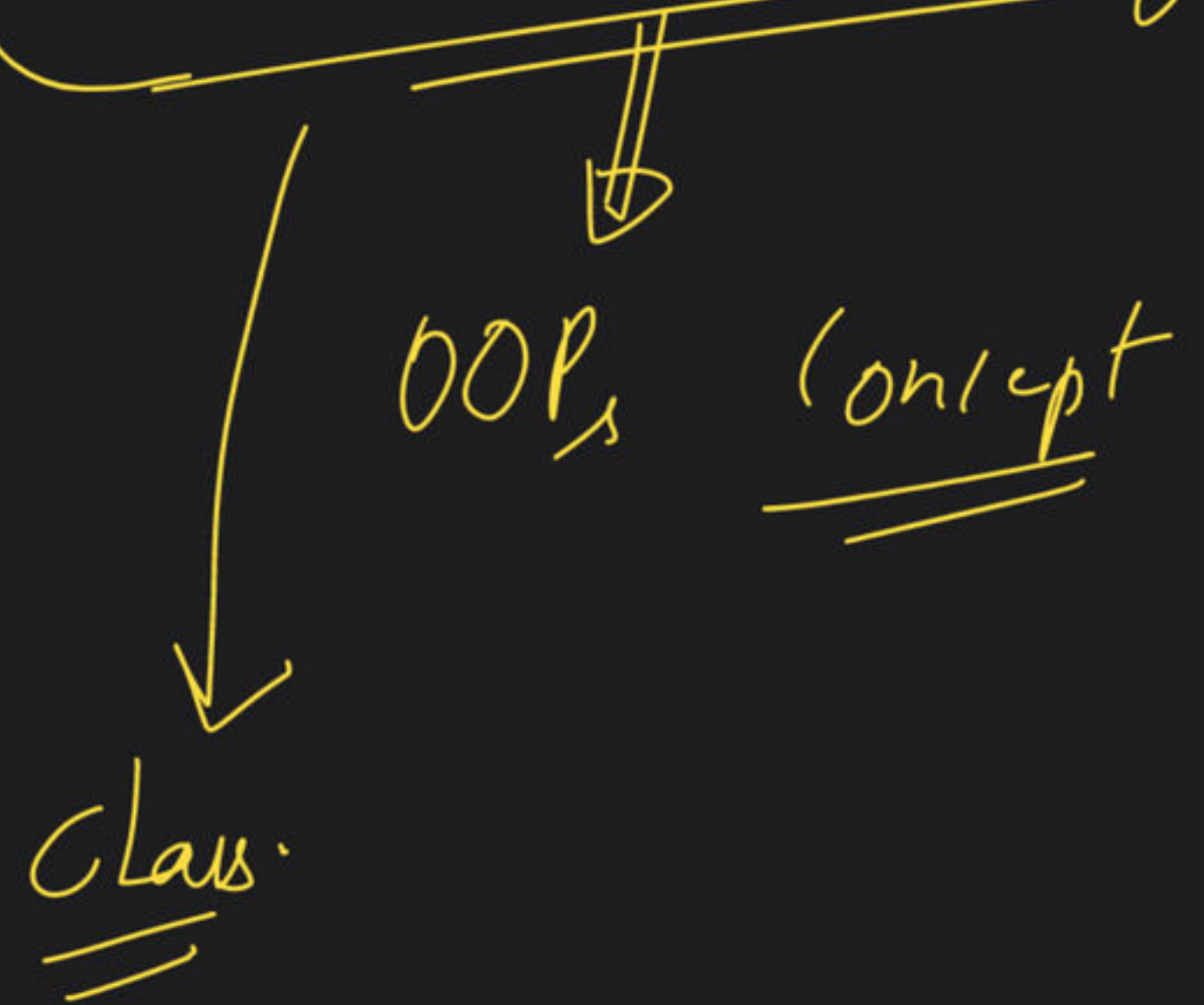
pointer

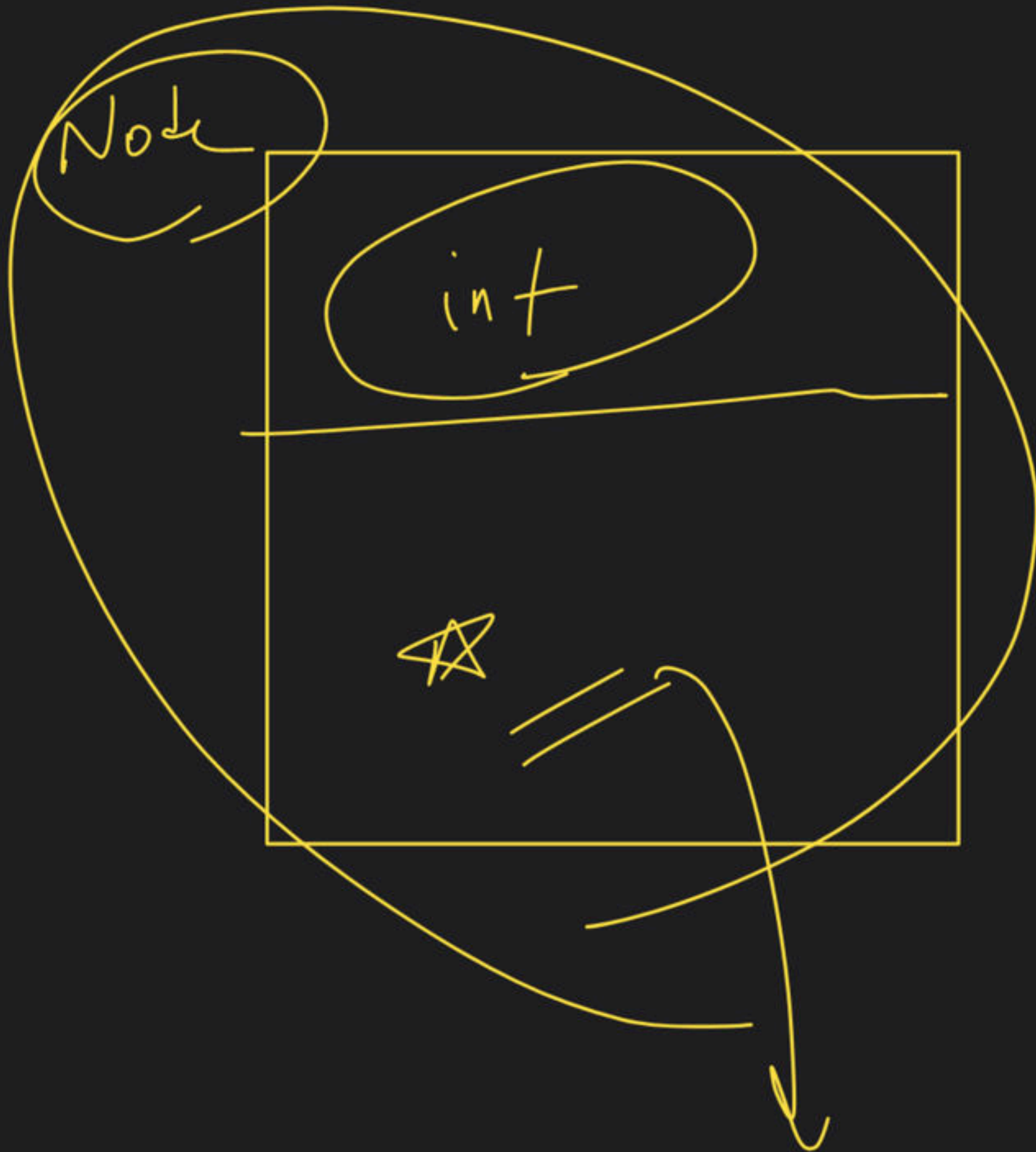
Node * next

Datatypes



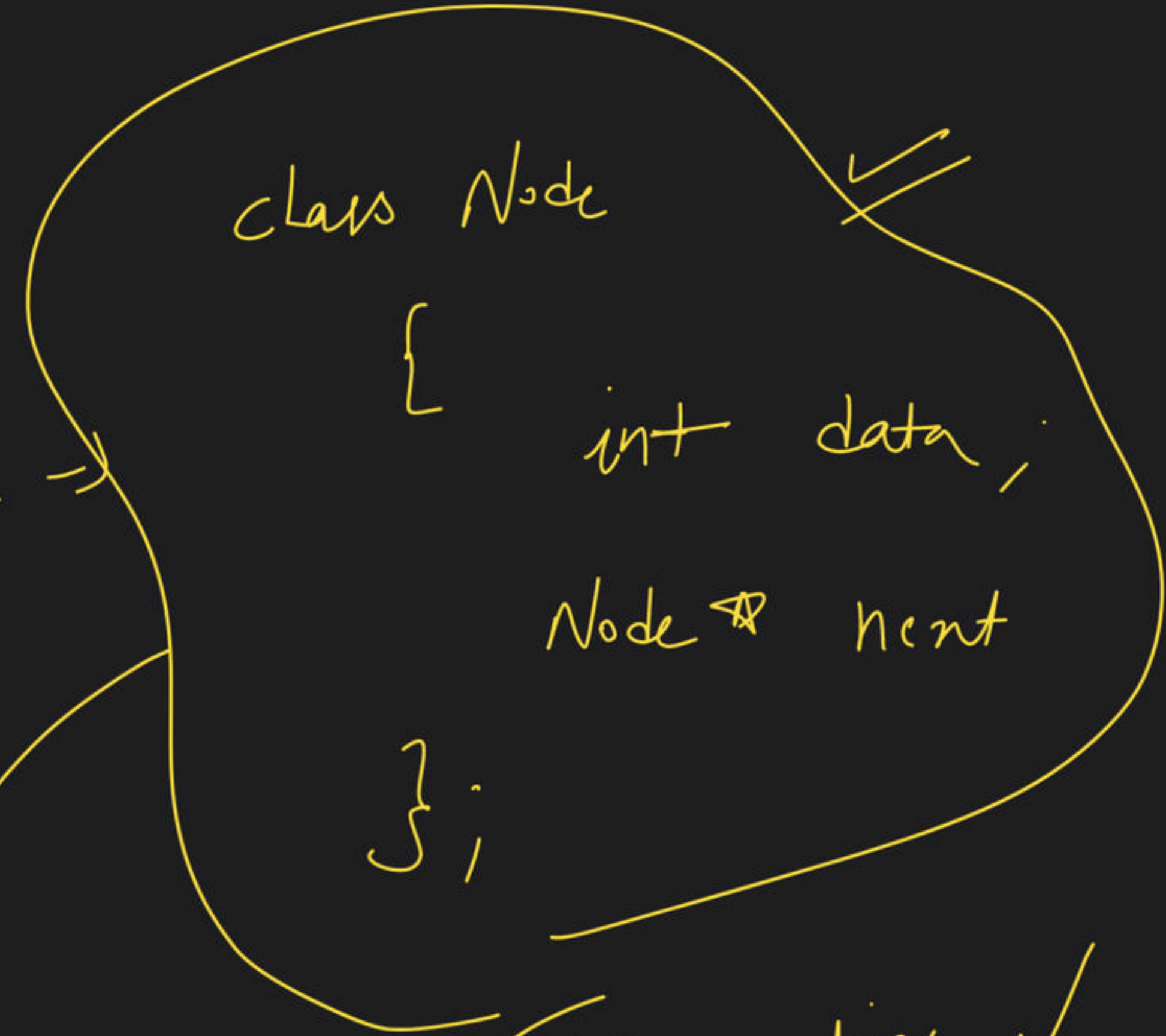
Custom data type





Node * a = new Node();

→
code →

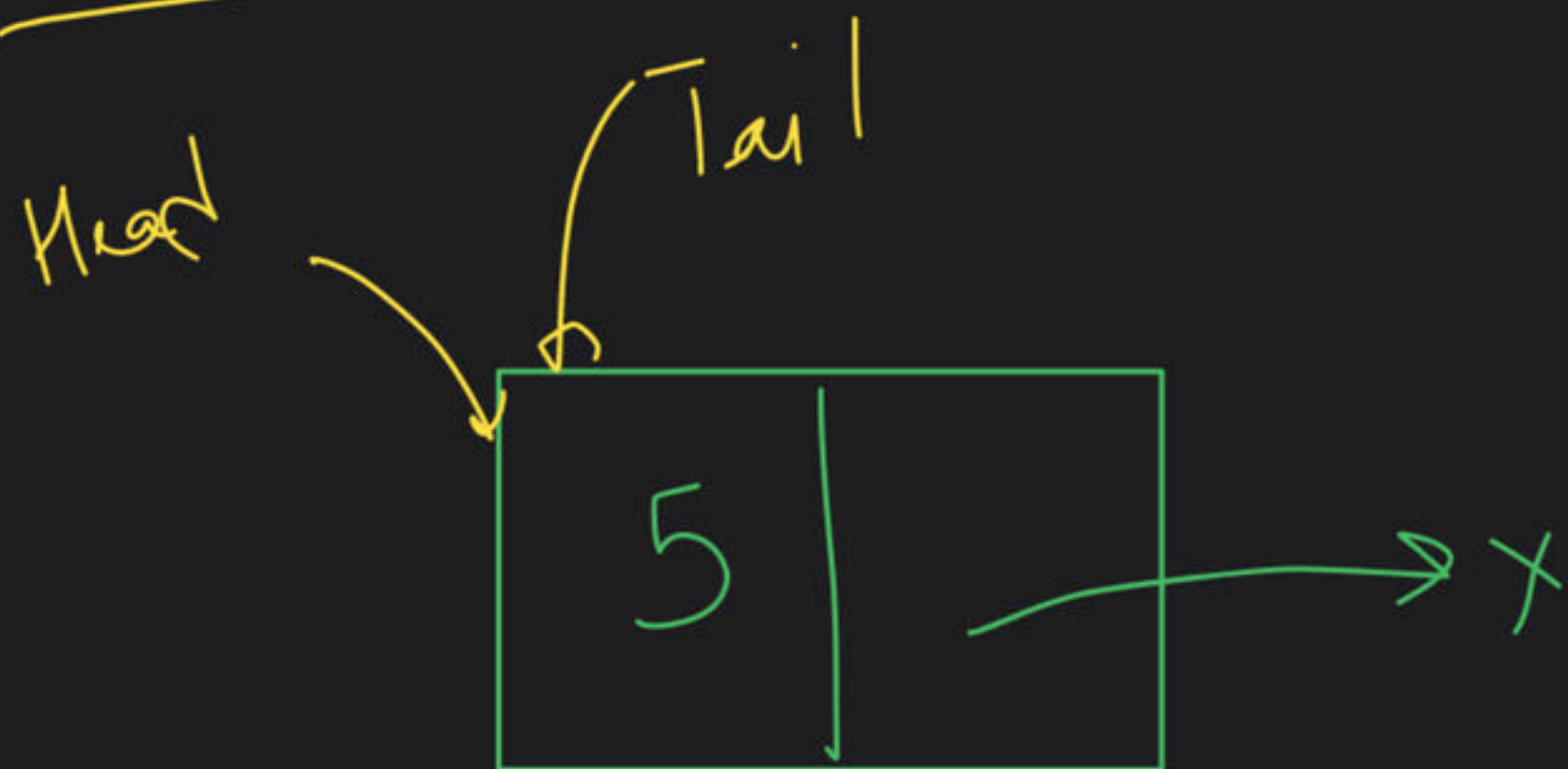
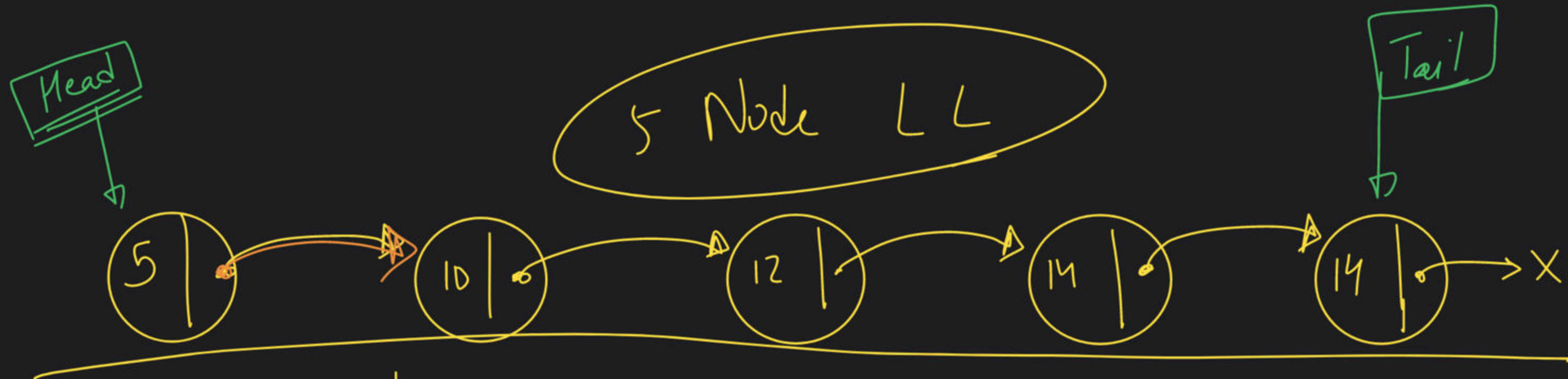


object

Static creation

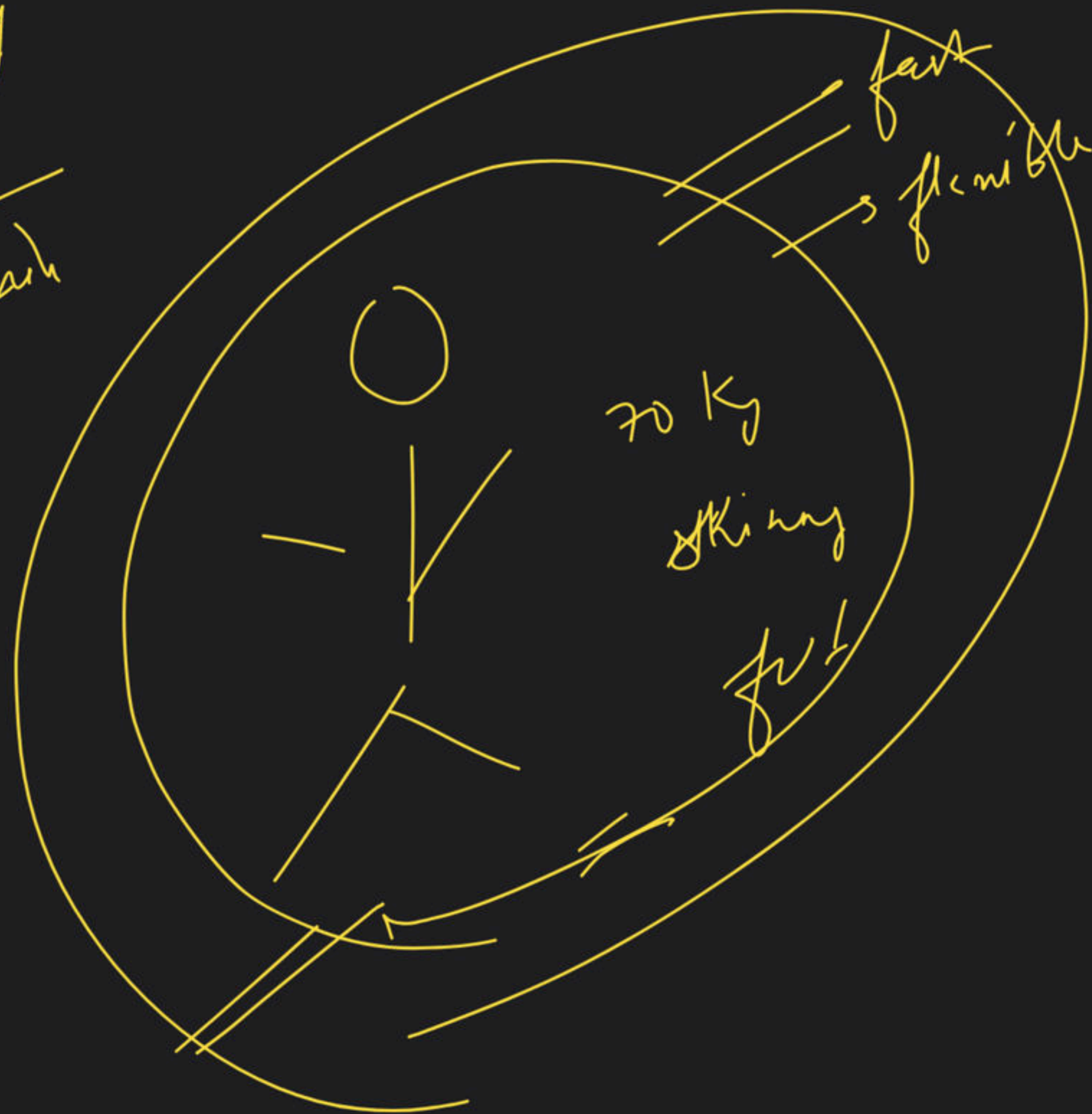
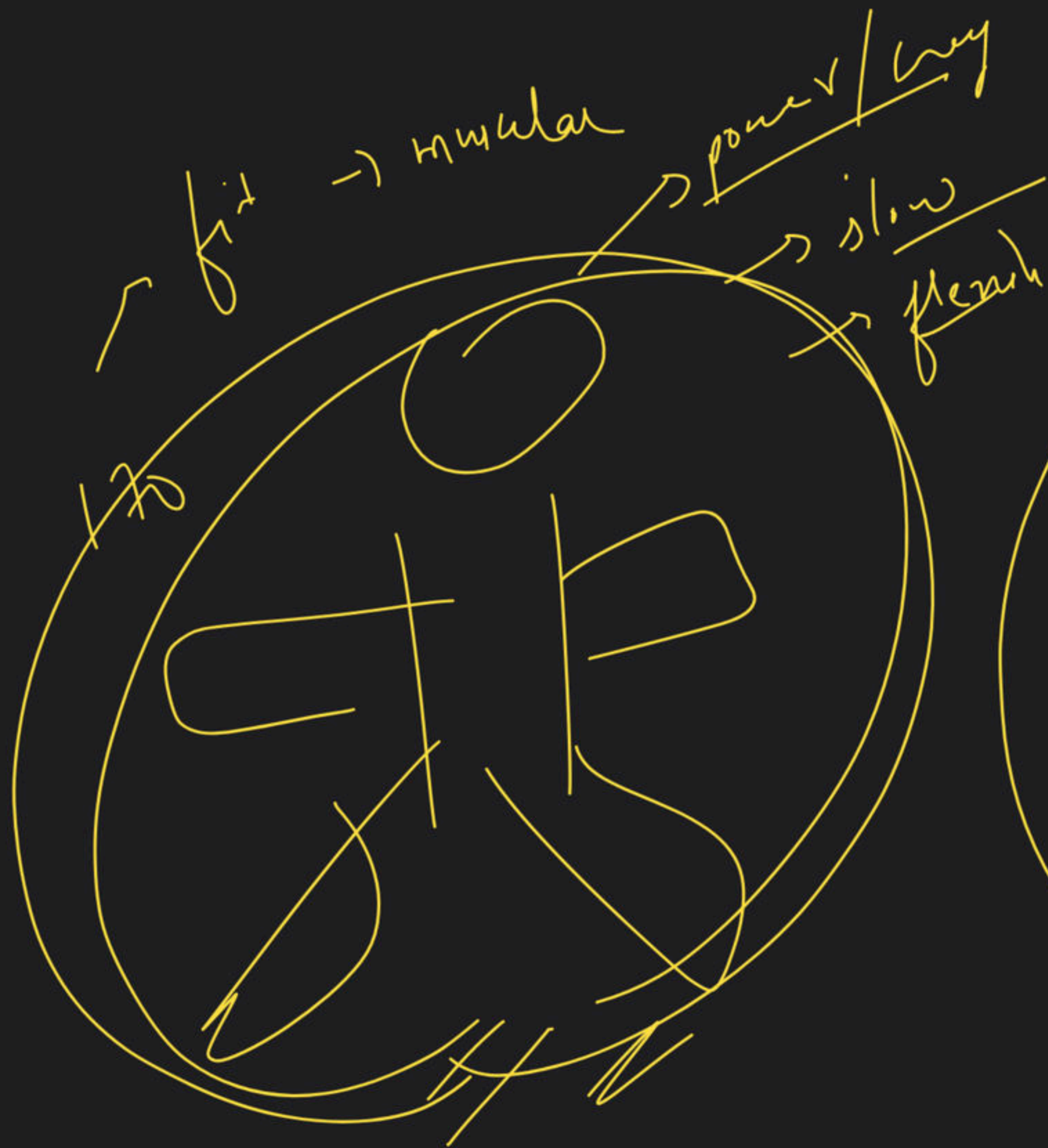
Node *

dynamic creation



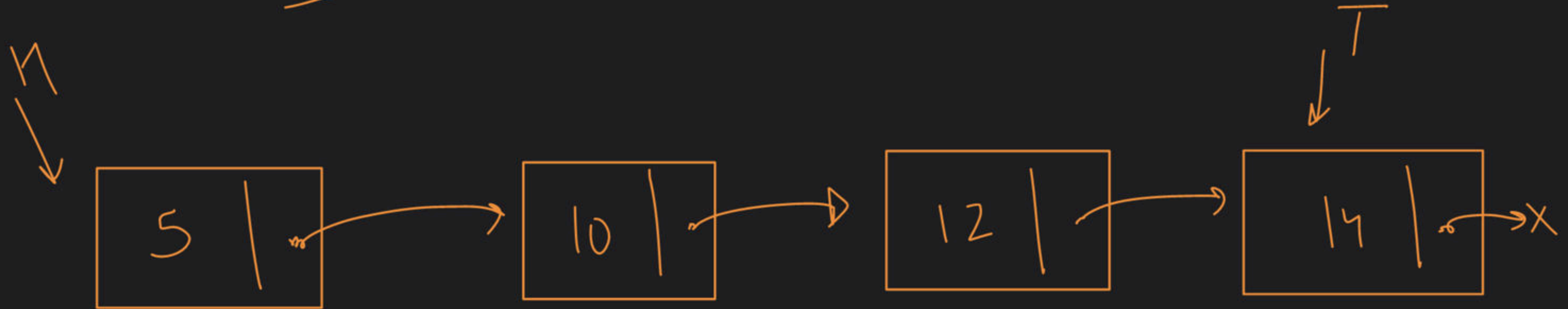
Single Node L L



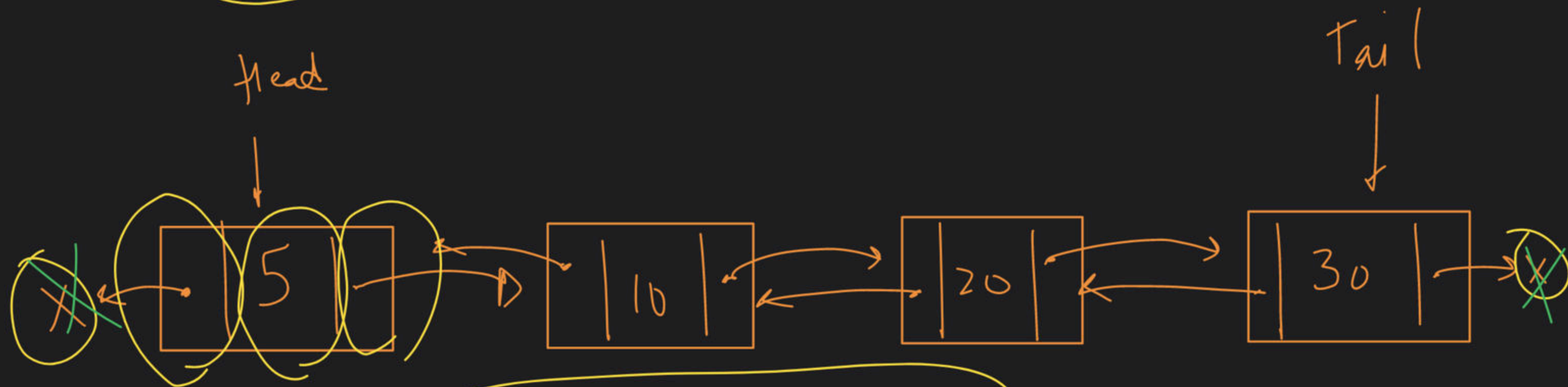


Type of Linked List

① Singly L.L

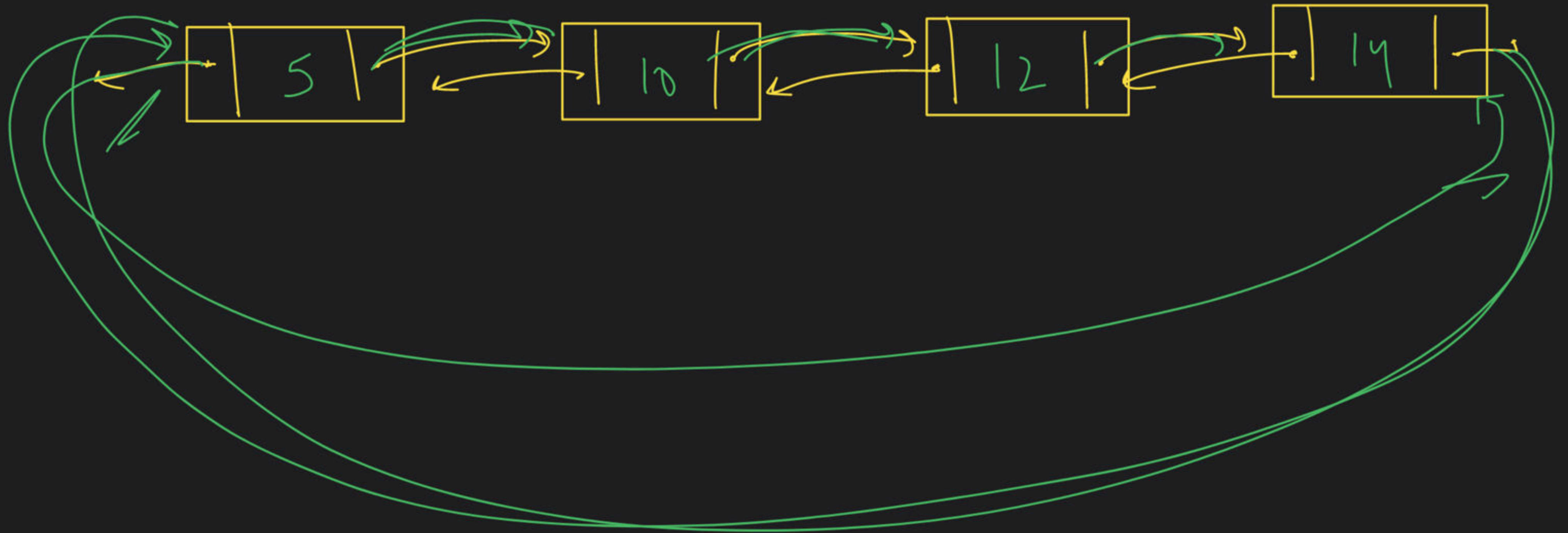


② Doubly Linked List



```
Node { Node *prev;  
      int data;  
      Node *next;  
}
```

(3) Circular Doubly Linked List



Example

add to queue

playlist

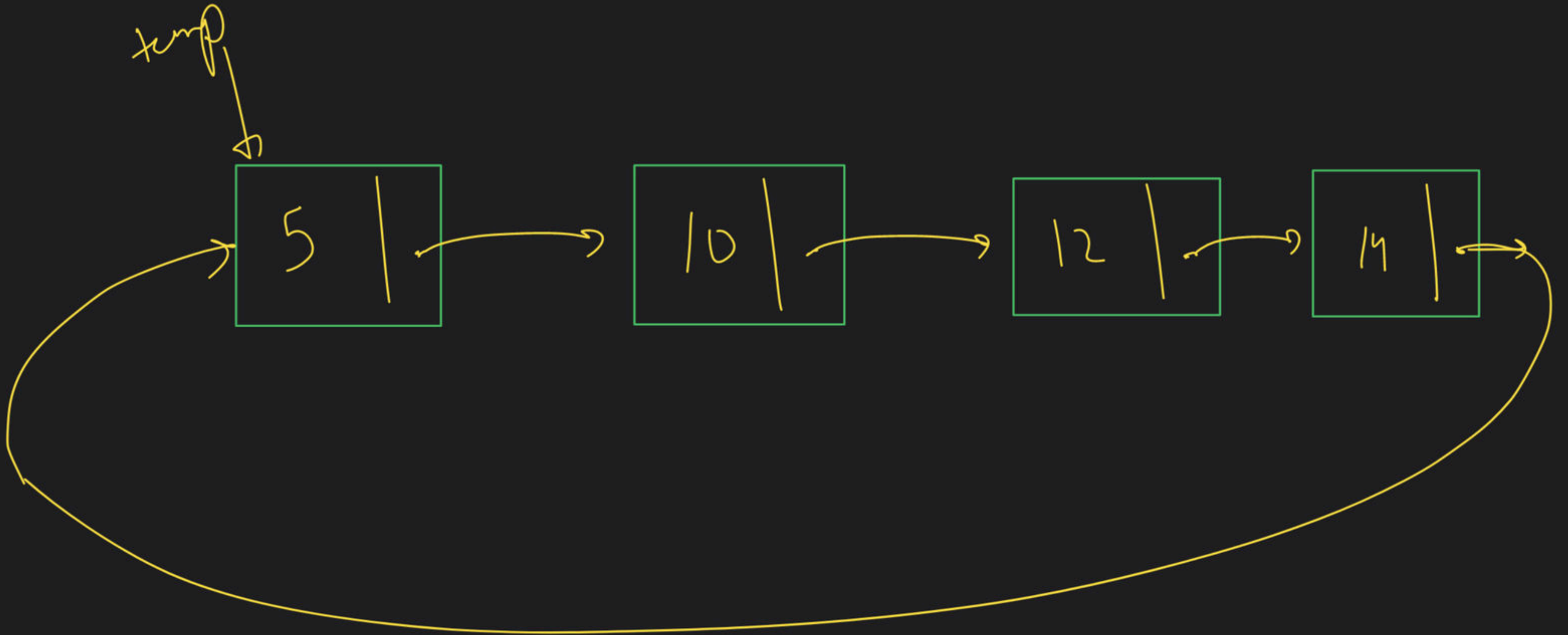
OS → memory mgmt

undo/redo

stacks/queue

④

Circular linked List



Singly linked list

↓
1 2 is Hindi

4/w {
→ length
→ main
→ mix
→ odd
→ even

Target

→ Creation

→ Insertion

→ Head

→ Tail

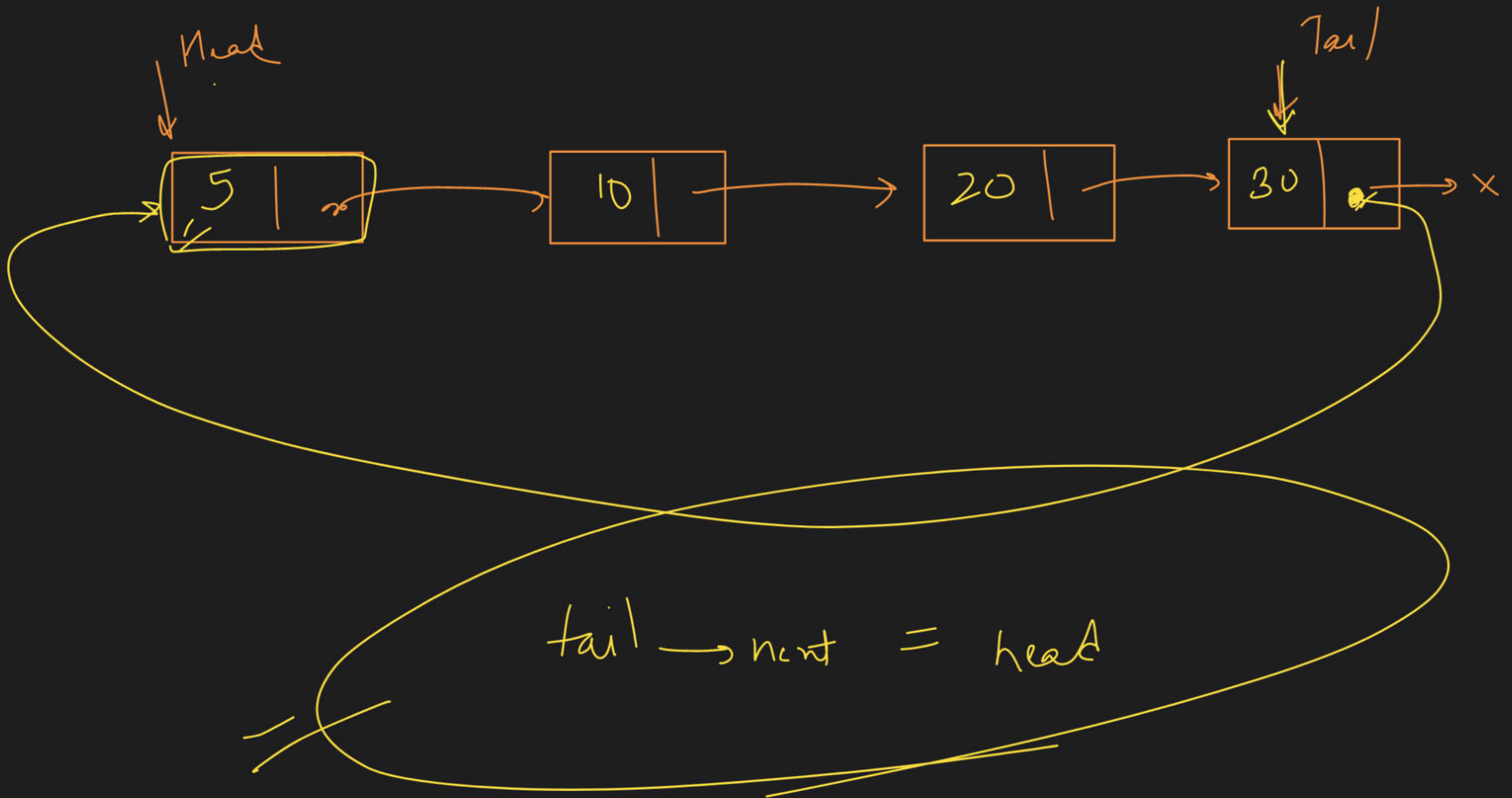
→ Randomly

→ Traversal

→ Searching

→ conversion to CLL

→ deletion

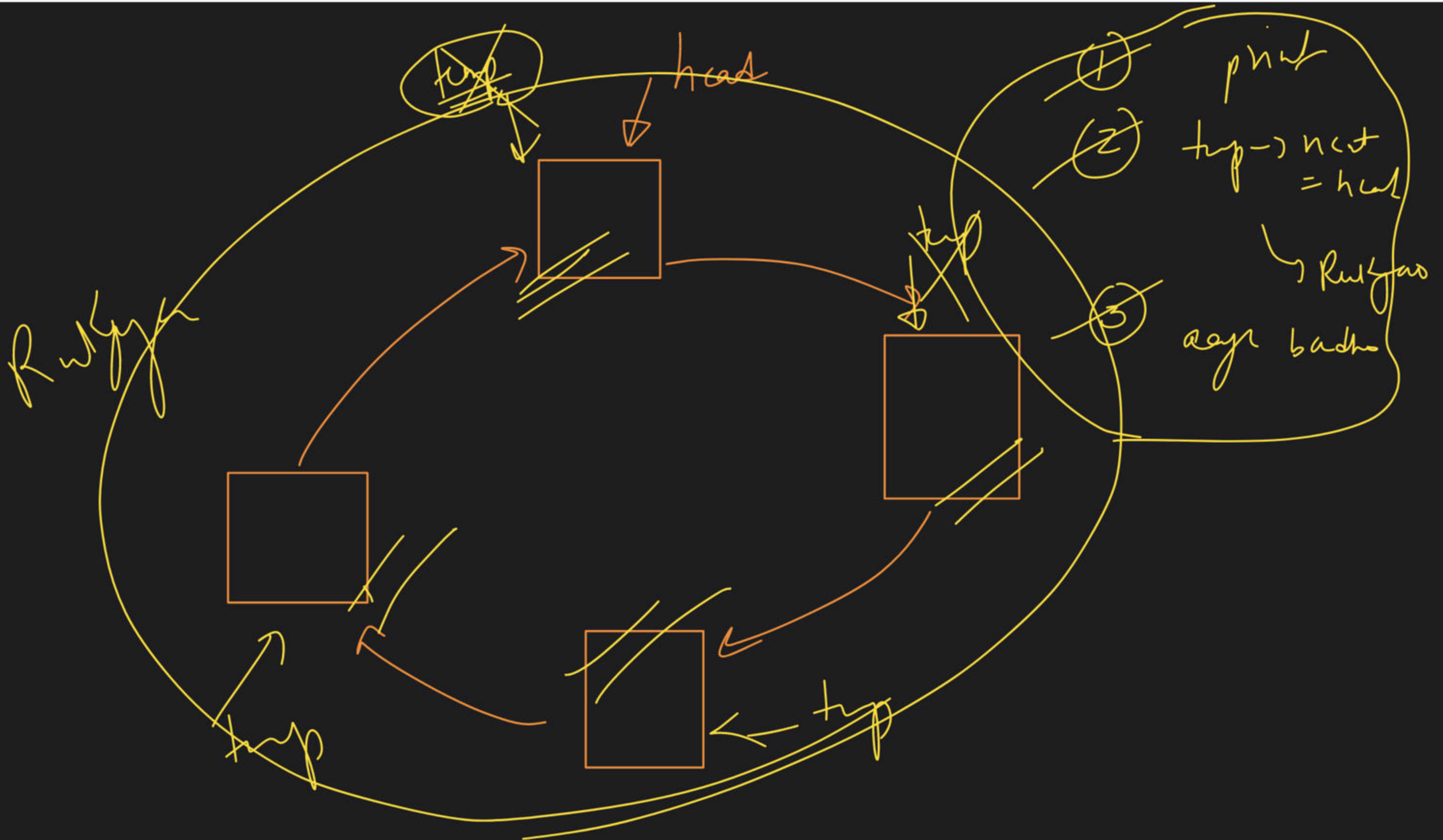


if (head == NULL)

do nothing

else

tail->next = head



while (~~temp~~ → next != ~~NULL~~)

head

cout << temp → data;

~~temp~~ → next

}

while (1)

~~{~~
~~low <= high → data;~~

if (temp → next = head)
break;

high = temp → next;

}

Insert At Position



$$\text{pos} = 3$$

$$\text{pos} - 2 = 3 - 2 = 1$$

$\text{pos} = 3$
 $\text{val} = 100$

(2) Create Node

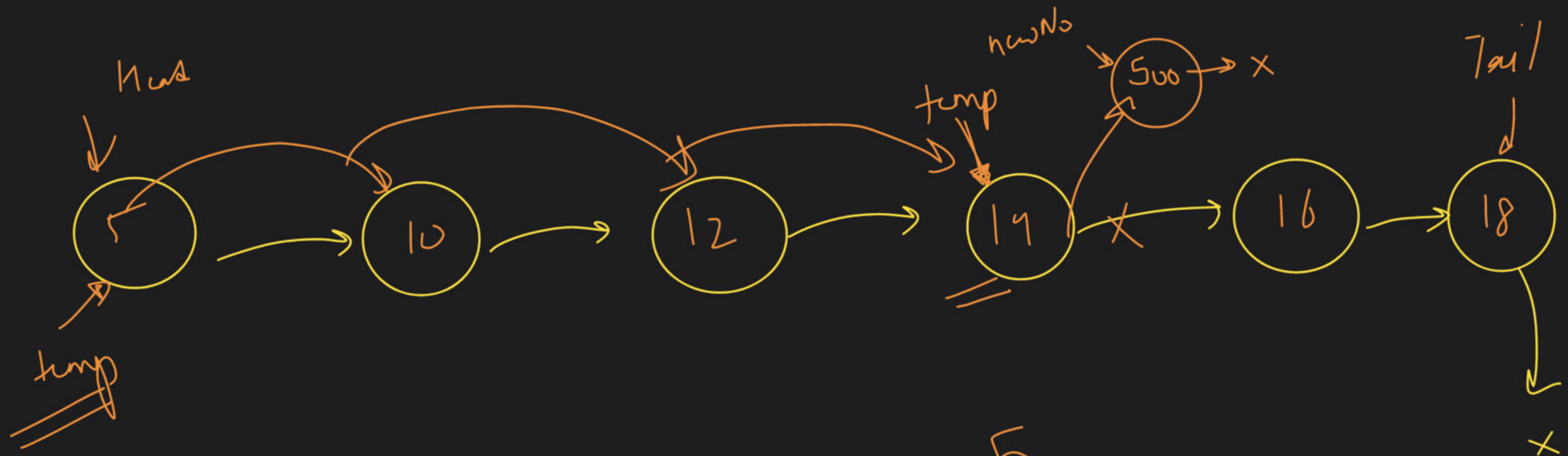
(1) reach at $\text{pos} - 1$

(3) $\text{newNode} \rightarrow \text{next} = \text{temp} \rightarrow \text{next}$

(4) $\text{temp} \rightarrow \text{next} = \text{newNode}$

if $(\text{pos} == 1)$:-
 $\rightarrow \text{insert At Head}(\text{val})$

if $(\text{pos} == \text{len} + 1)$
 $\rightarrow \text{insert At Tail}(\text{val})$



pos = 5
Val = 500

pos = 5
pos - 2 jump

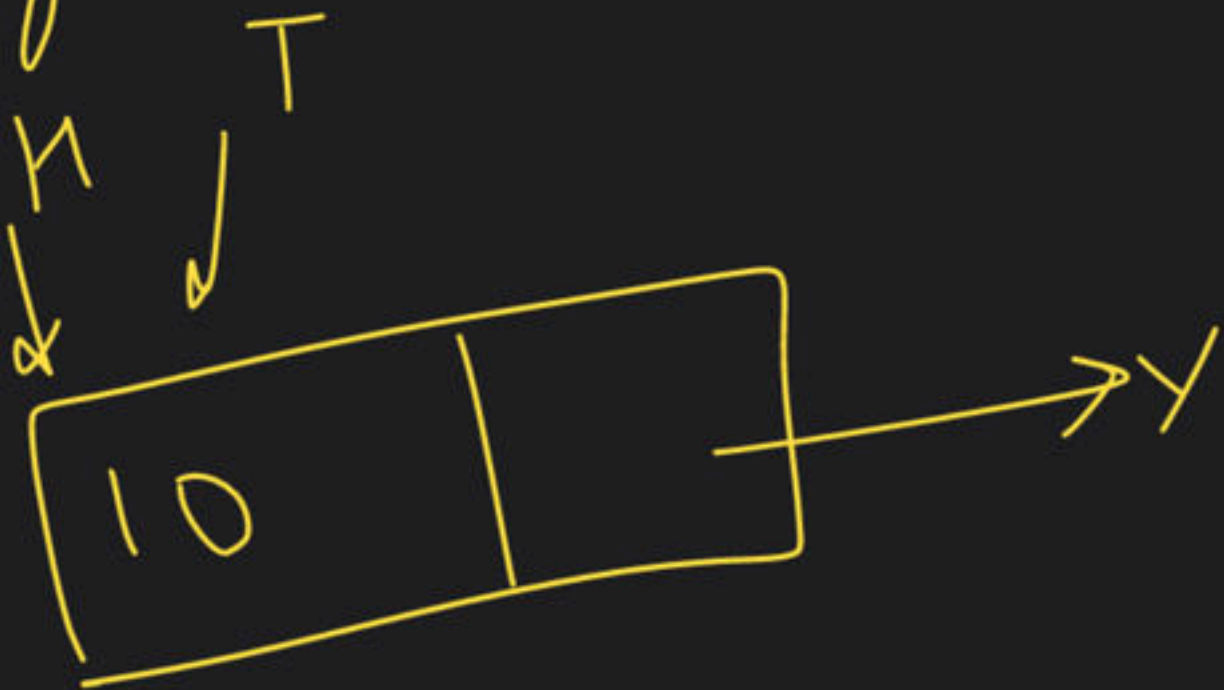
delete from position (5)

(1) Empty



if (head == NULL & tail == NULL)
return;

(2) Single Node



if (len < Pos)
return;

delete from pos (1)

Single Node



✓ head = NULL

✓ tail = NULL

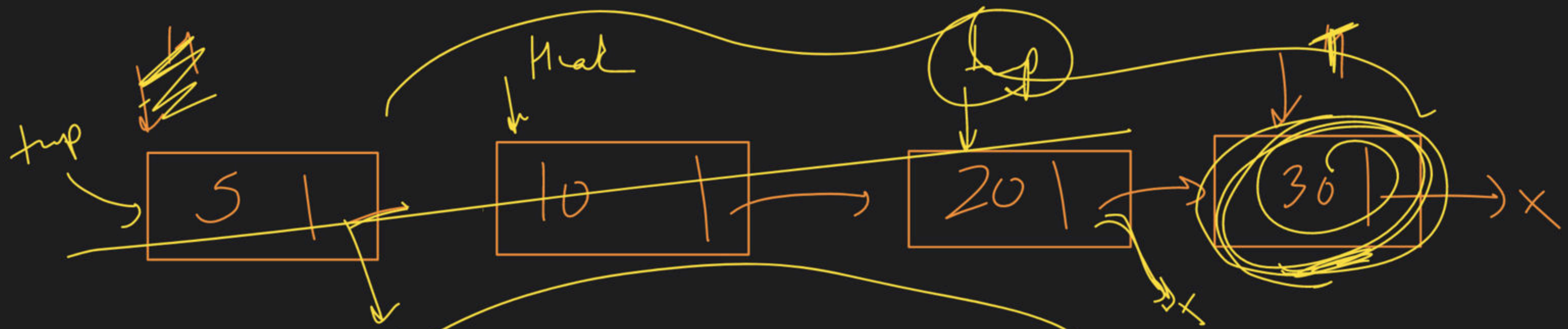
delete temp

Node * temp = head

head = NULL

tail = NULL

~~delete temp~~



delete first node

Node * temp = head

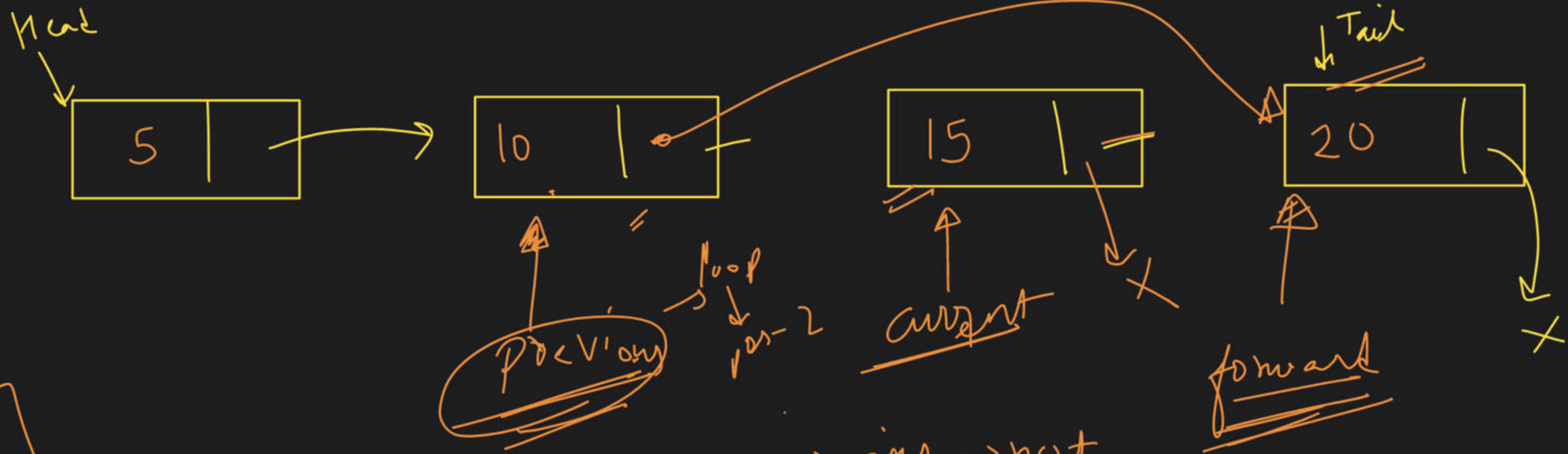
head = head -> next

temp -> next = NULL

delete temp

delete last node

OK



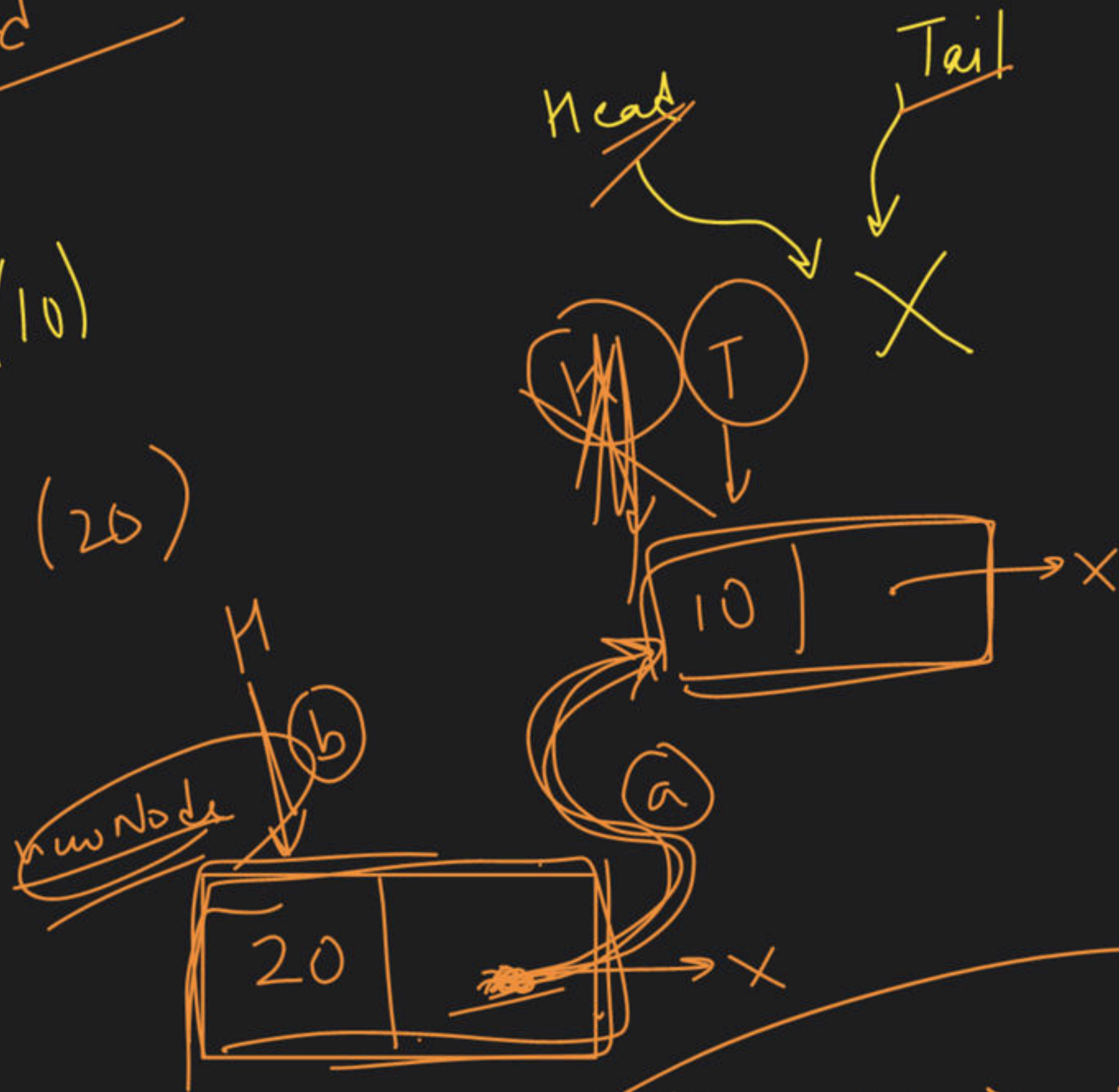
pos = 3
↓
delete

- ① previous / current / forward → setup
- ② current → next = NULL
- ③ previous → next = forward
- ④ delete current

→ Insert At Head

① Insert At Head (10)

② Insert At Head (20)



- (a) $\text{newNode} \rightarrow \text{next} = \text{head}$
- (b) $\text{head} = \text{newNode}$

Empty LL

- ① Node Create
 newNode

- ② $\text{head} = \text{newNode}$
 $\text{tail} = \text{newNode}$

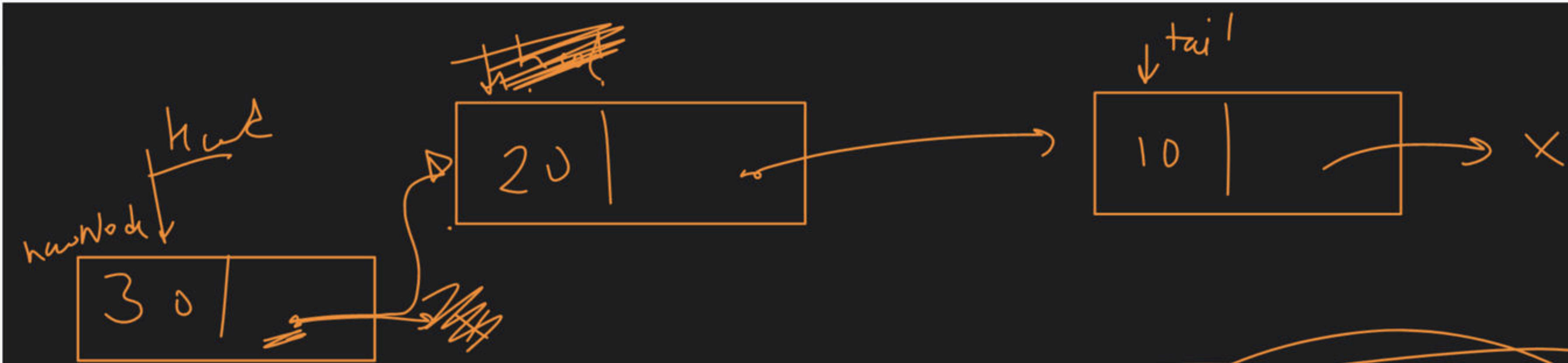
① Create Node
 newNode



Insert At Head

Empty LL → ?

Non Empty LL → ?



Insert At Head (30)

Node ~~A~~ newNode = new Node(30)

newNode → next = head

head = newNode

pointer $\rightarrow$ data member

newNode $\rightarrow$ next

newNode $\rightarrow$ data

```
if (head == NULL & tail == NULL)
{
    Node * newNode = new Node (data);
    head = newNode;
    tail = newNode;
}
```

```
else
{
    Node * newNode = new Node (data);
    newNode -> next = head;
    head = newNode;
}
```

Class
object
pointer
construct
dynamic
allocation

① Empty LL

Head $\longrightarrow$ NULL
Tail $\longrightarrow$ NULL

② Insert At Head (5)



Node * newNode = new Node(5);



head = newNode

tail = newNode

③ Insert At Head (10)



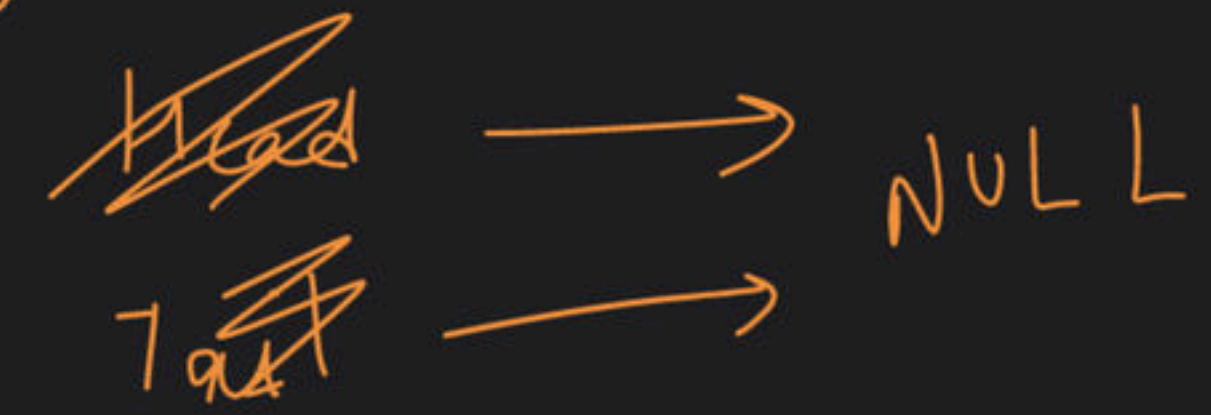
Node * newNode = new Node(10);

newNode $\rightarrow$ next = head

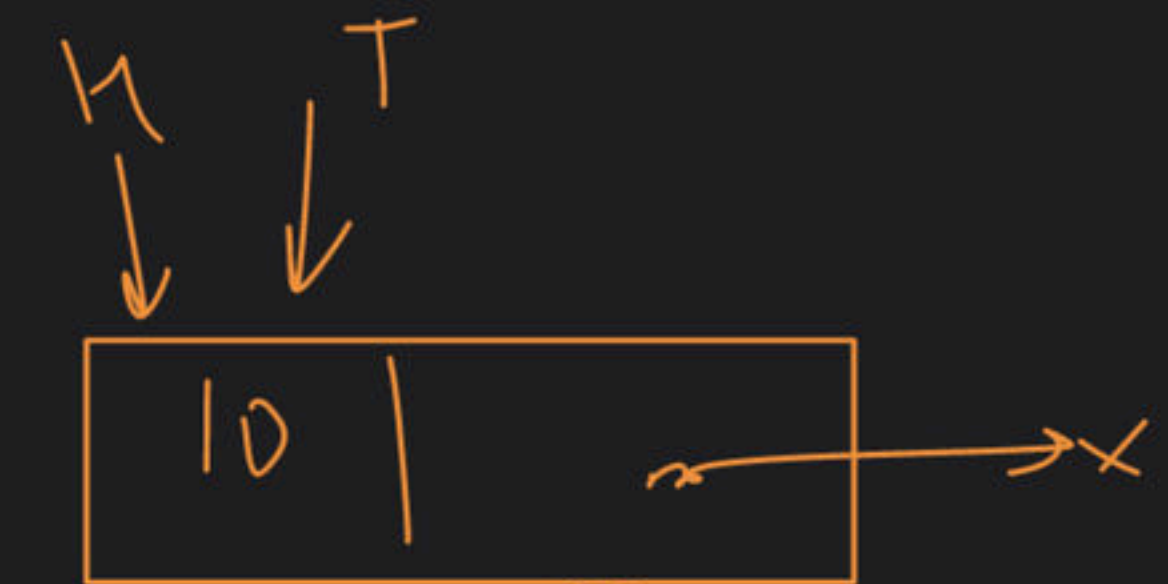
head = newNode

Insert At Tail

(1) Empty list



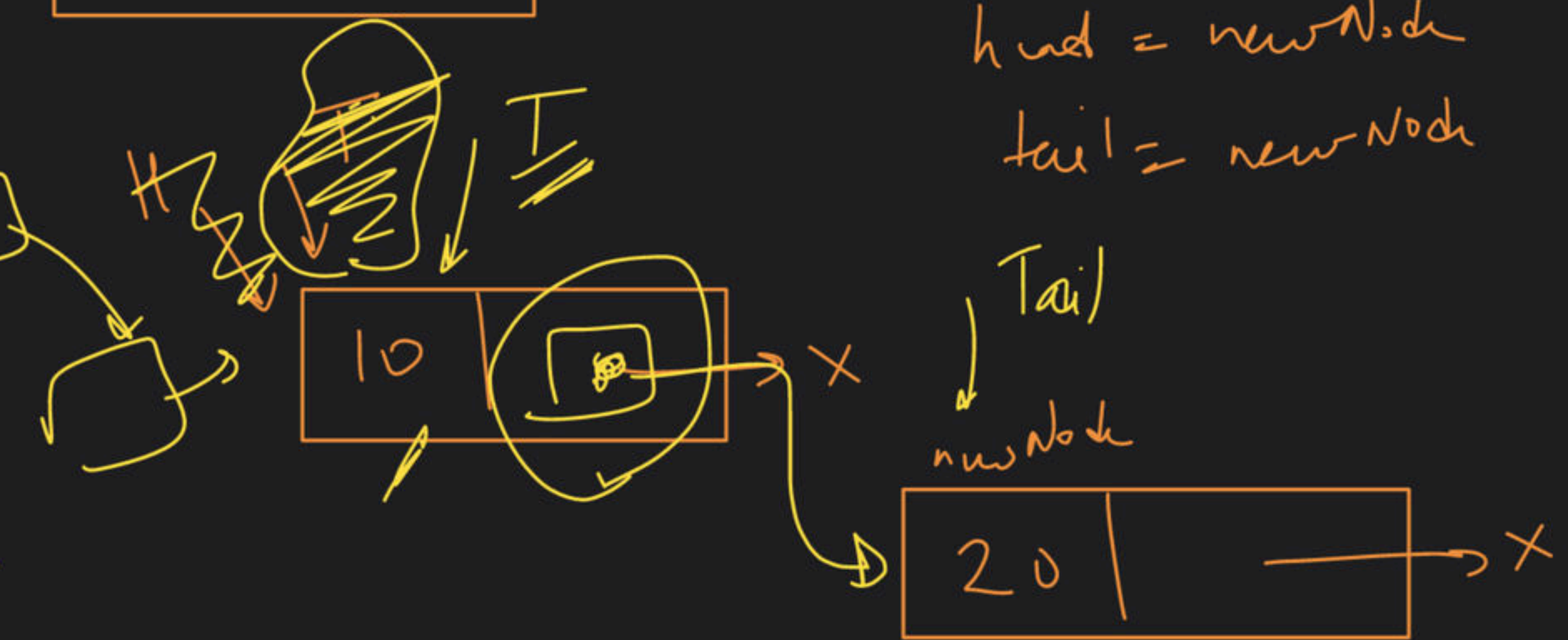
Insert At Tail (10)



Node *newNode = new Node(10);
 head = newNode
 tail = newNode

Insert At Tail (20)

Node *newNode = new Node(20);
 tail->next = newNode
 tail = newNode



Insert At Tail (30)

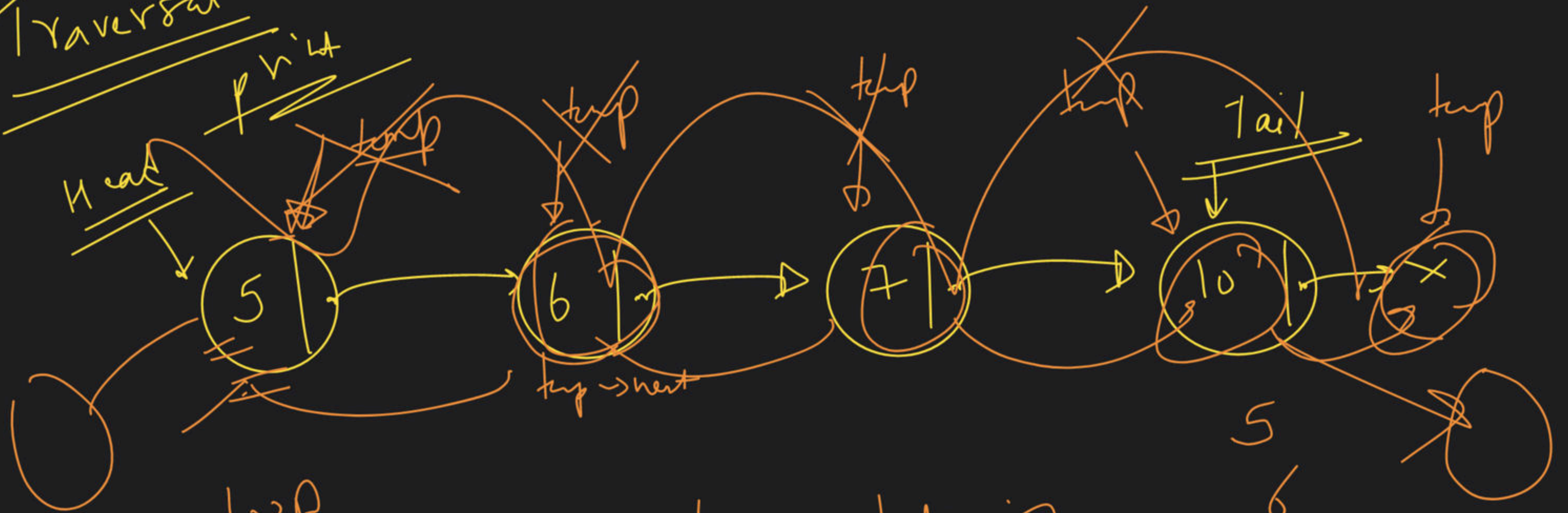


Node * newNode = new Node (30);

tail->next = newNode

tail = newNode

Traversal



loop
cout << temp -> data;
temp = temp -> next
}
mukha kha
hai
temp = NULL

5
6
7
10

Searching

→ Target → T/F

2 min

5



10



15



Target → 15

→ True

Target → 17

→ false

Ww

min

max

odd

even

sum

